



Université Cadi Ayyad
École Normale Supérieure de Marrakech
Département d'Informatique

Licence d'éducation : Spécialité Enseignement Secondaire — Informatique
Semestre S6
Module : Développement Web et Mobile 2

Rapport de Projet de Fin de Module
*Conception et développement d'une application web
et mobile de covoiturage urbain*

Réalisé par :
Majjati Mohamed Taha

Encadré par :
Pr. Mohamed LACHGAR

Année universitaire : 2025–2026

Déclaration

Je soussigné, **Majjati Mohamed Taha**, étudiant à l'École Normale Supérieure de Marrakech, Université Cadi Ayyad, Département d'Informatique, déclare que le présent rapport constitue le fruit de mon travail personnel. Les sources externes utilisées ont fait l'objet de citations et références explicites. Tout manquement à cet engagement relèverait du plagiat, considéré comme une infraction académique passible de sanctions.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques, au bénéfice des futurs étudiants, et j'accepte qu'il soit consulté dans l'intérêt général de l'Enseignement Supérieur.

Majjati Mohamed Taha
Marrakech, 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à Monsieur Lachgar pour l'encadrement rigoureux et bienveillant dont il m'a fait bénéficier tout au long de la réalisation de ce projet.

Sa disponibilité, son écoute attentive et la pertinence de ses conseils ont grandement contribué à la qualité de ce travail. Son soutien constant et ses remarques constructives ont permis de progresser et de mener ce projet à son terme dans les meilleures conditions.

Qu'il trouve ici l'expression de ma sincère reconnaissance et de mon profond respect.

Conception et développement d'une application web et mobile.

Majjati Mohamed Taha

Abstract

This project presents the design and development of an urban carpooling platform targeting the daily commute needs of students, teachers and staff at École Normale Supérieure de Marrakech. The absence of a digital coordination tool between drivers and passengers leads to wasted resources, increased traffic and a poor user experience on and around campus.

The system is composed of three interconnected components : an **Android mobile application** for both drivers and passengers, a **Node.js REST API backend** providing secure data management, and a **React web dashboard** for administrative supervision. The database is structured around four core tables : *users*, *rides*, *ride_bookings* and *ride_reviews*.

Key features include ride publication and search, reservation management with status tracking (pending, accepted, refused), a rating and review system, and a comprehensive admin dashboard with statistics.

Keywords : Carpooling, Android Java, React.js, Node.js, MySQL, REST API, JWT, Dashboard, Volley.

Résumé

Ce projet porte sur la conception et le développement d'une plateforme de covoiturage urbain destinée aux étudiants, enseignants et personnels de l'École Normale Supérieure de Marrakech. L'objectif est de faciliter l'organisation des trajets domicile-campus en mettant en relation des conducteurs et des passagers via une solution numérique simple et fiable.

La solution repose sur trois composants complémentaires : une **application mobile Android** destinée aux conducteurs et aux passagers, une **API REST Node.js** assurant la gestion sécurisée des données, et une **interface web React** dédiée à la supervision administrative. La base de données MySQL est structurée autour de quatre tables : *users*, *rides*, *ride_bookings* et *ride_reviews*.

Les fonctionnalités principales incluent la publication et la recherche de trajets, la réservation de places avec suivi des statuts, un système d'avis et de notes, ainsi qu'un tableau de bord administrateur.

Mots-clés : Covoiturage, Android Java, React.js, Node.js, MySQL, API REST, JWT, Dashboard, Volley.

Liste des abréviations

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
REST	Representational State Transfer
SQL	Structured Query Language
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
URL	Uniform Resource Locator
BDD	Base de Données
MVC	Modèle-Vue-Contrôleur

Table des figures

1.1	Diagramme de Gantt du projet	5
3.1	Acteurs principaux du système de covoiturage	9
3.2	Fonctionnalités principales du système de covoiturage	10
3.3	Processus métier du système de covoiturage	12
3.4	Diagramme de cas d'utilisation du système de covoiturage	13
4.1	Architecture globale du système de covoiturage	14
4.2	Diagramme de classes du système de covoiturage	16
4.3	Diagramme de séquence de réservation d'un trajet	17
5.1	Dashboard administrateur — Page de connexion	21
5.2	Dashboard administrateur — Vue générale des statistiques	21
5.3	Dashboard administrateur — Liste des trajets	22
5.4	Dashboard administrateur — Gestion des réservations	22
5.5	Dashboard administrateur — Gestion des avis	23
5.6	Application Android — Écran de connexion	24
5.7	Application Android — Publication d'un trajet	25
5.8	Application Android — Recherche et réservation d'un trajet	26

Liste des tableaux

1.1	Les acteurs du système	4
1.2	Planification des sprints	4
2.1	Positionnement du projet par rapport aux solutions existantes	7
3.1	Les acteurs du système et leurs fonctionnalités	8
4.1	Structure des tables de la base de données	15
4.2	Endpoints API principaux	18
5.1	Technologies utilisées et justification	19
5.2	Résultats des tests API (Postman)	26

Table des matières

Introduction générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Présentation du projet	2
1.2.1 Sujet du projet	2
1.2.2 Intérêt du projet	2
1.3 Problématique et état de l'existant	3
1.4 Objectifs du projet	3
1.4.1 Objectifs fonctionnels	3
1.4.2 Objectifs techniques	3
1.4.3 Objectifs organisationnels	3
1.5 Acteurs du système	4
1.6 Méthodologie adoptée	4
1.6.1 Planification du projet	4
1.7 Conclusion	5
2 État de l'art	6
2.1 Introduction	6
2.2 Le covoiturage urbain : définition et enjeux	6
2.3 Solutions existantes	6
2.3.1 Applications grand public de covoiturage	6
2.3.2 Applications de transport urbain	6
2.4 Comparaison des solutions	7
2.5 Limites des solutions existantes	7
2.6 Positionnement du projet	7
2.7 Conclusion	7
3 Analyse fonctionnelle	8
3.1 Introduction	8
3.2 Identification des acteurs	8
3.3 Besoins fonctionnels	10
3.3.1 Besoins du conducteur	10
3.3.2 Besoins du passager	10
3.3.3 Besoins de l'administrateur	11
3.3.4 Besoins du système (Backend API)	11
3.4 Besoins non fonctionnels	11
3.5 Règles métier	11
3.6 Scénarios d'utilisation	12
3.6.1 Scénario 1 : Publication d'un trajet par un conducteur	12
3.6.2 Scénario 2 : Réservation d'une place par un passager	12

3.6.3	Scénario 3 : Supervision par l'administrateur	12
3.7	Diagramme de cas d'utilisation	13
3.8	Conclusion	13
4	Conception du système	14
4.1	Introduction	14
4.2	Architecture générale	14
4.3	Modèle de données	15
4.4	Diagramme de classes	15
4.5	Diagrammes de séquence	17
4.5.1	Réservation d'un trajet	17
4.5.2	Réservation d'une place	17
4.6	Conception de l'API REST	18
4.7	Sécurité du système	18
4.8	Conclusion	18
5	Réalisation du projet	19
5.1	Introduction	19
5.2	Environnement de développement	19
5.3	Choix technologiques	19
5.4	Réalisation du backend	19
5.4.1	Structure du projet	19
5.4.2	Middleware d'authentification JWT	20
5.5	Réalisation de l'interface web (Dashboard)	21
5.5.1	Page de connexion	21
5.5.2	Tableau de bord	21
5.5.3	Gestion des trajets	22
5.5.4	Gestion des réservations	22
5.5.5	Gestion des avis	22
5.6	Réalisation de l'application mobile	23
5.6.1	Structure du projet Android	23
5.6.2	Écran de connexion	23
5.6.3	Publication d'un trajet (conducteur)	24
5.6.4	Recherche et réservation (passager)	25
5.7	Tests et validation	26
5.8	Conclusion	27
	Conclusion générale	28

Introduction générale

Dans un contexte universitaire en constante évolution, la mobilité quotidienne entre le domicile et le campus constitue un enjeu majeur pour les étudiants, les enseignants et le personnel de l'École Normale Supérieure de Marrakech. L'absence d'un outil numérique de coordination engendre une mauvaise utilisation des véhicules disponibles, une augmentation du trafic aux abords du campus et une expérience utilisateur limitée.

C'est dans cette optique que s'inscrit le présent projet, réalisé dans le cadre du module *Développement Web et Mobile 2* du Département d'Informatique. L'objectif est de concevoir et développer une plateforme complète de covoiturage urbain, permettant de mettre en relation des conducteurs et des passagers effectuant des trajets domicile-campus.

La solution développée repose sur trois composants complémentaires :

- Une **application mobile Android** permettant aux conducteurs de publier des trajets et aux passagers de les rechercher, réserver et évaluer.
- Une **API REST** développée avec Node.js et Express, servant de couche centrale de communication entre les clients et la base de données MySQL.
- Une **interface web React** offrant aux administrateurs une supervision complète du système via un tableau de bord.

Ce rapport est structuré en cinq chapitres :

1. **Contexte général du projet** — présentation du problème, objectifs et méthodologie
2. **État de l'art** — étude des solutions existantes et positionnement du projet
3. **Analyse fonctionnelle** — besoins, acteurs et cas d'utilisation
4. **Conception du système** — architecture, modèle de données et diagrammes UML
5. **Réalisation du projet** — implémentation, interfaces et tests

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre présente le contexte général du projet de covoiturage urbain développé pour le campus de l'École Normale Supérieure de Marrakech. Il vise à situer le projet dans son environnement, en mettant en évidence les enjeux liés à la mobilité domicile-campus, les limites des solutions existantes, ainsi que les objectifs et la méthodologie adoptée.

1.2 Présentation du projet

1.2.1 Sujet du projet

Le projet consiste en la conception et le développement d'une **application de covoiturage urbain** permettant de connecter des conducteurs et des passagers effectuant des trajets quotidiens domicile-campus au sein de la communauté ENS Marrakech.

Ce système repose sur trois composants principaux :

- Une **application mobile Android** destinée aux utilisateurs (conducteurs et passagers)
- Un **backend Node.js/Express** assurant la gestion des données et la logique métier via une API REST
- Un **dashboard web React** permettant à l'administrateur de superviser le système

Le système permet notamment :

- La publication de trajets par les conducteurs (ville de départ, destination, date, prix, places)
- La recherche et la réservation de places par les passagers
- La gestion des réservations avec suivi des statuts (en attente, acceptée, refusée)
- L'ajout d'avis et d'évaluations après participation
- Le suivi global via un tableau de bord administrateur avec statistiques

1.2.2 Intérêt du projet

Ce projet présente plusieurs avantages directs pour la communauté ENS Marrakech :

- **Optimisation des déplacements** : réduction du nombre de véhicules en circulation grâce au partage de trajets domicile-campus
- **Réduction des coûts** : partage des frais de transport entre conducteurs et passagers
- **Amélioration de l'expérience utilisateur** : interface simple et intuitive adaptée aux étudiants et enseignants
- **Aide à la gestion** : tableau de bord permettant à l'administration de suivre les trajets, réservations et avis

- **Sécurité et fiabilité** : système d'authentification JWT sécurisé, validation des données et gestion des statuts

1.3 Problématique et état de l'existant

Au sein du campus ENS Marrakech, aucun système numérique n'existe actuellement pour coordonner les trajets entre les membres de la communauté universitaire. Cette absence entraîne plusieurs difficultés concrètes :

- Manque de solutions adaptées aux trajets domicile-campus quotidiens
- Difficulté de coordination entre conducteurs et passagers
- Absence de système de gestion des réservations et des statuts
- Manque de suivi et de contrôle via une interface administrateur dédiée

Les solutions grand public telles que BlaBlaCar ne sont pas adaptées à ce contexte spécifique : elles ciblent les trajets longue distance, ne permettent pas la gestion d'une communauté fermée, et ne proposent pas d'interface d'administration pour une institution. Le projet proposé comble ce vide avec une solution sur mesure.

1.4 Objectifs du projet

1.4.1 Objectifs fonctionnels

- Permettre à un conducteur de publier un trajet en indiquant ville de départ, destination, date, heure, prix et nombre de places disponibles
- Permettre à un passager de rechercher et filtrer les trajets disponibles par ville de départ
- Permettre la réservation d'une place et le suivi du statut (en attente, acceptée, refusée)
- Permettre au conducteur d'accepter ou de refuser des demandes de réservation
- Permettre au passager de noter un trajet et de laisser un commentaire après participation
- Offrir à l'administrateur un tableau de bord de supervision avec statistiques

1.4.2 Objectifs techniques

- Concevoir une API REST sécurisée avec authentification JWT
- Implémenter une base de données relationnelle MySQL structurée autour de 4 tables : *users*, *rides*, *ride_bookings*, *ride_reviews*
- Développer une application Android Java avec Volley pour les appels HTTP
- Développer une interface web d'administration avec React
- Assurer la validation des données côté serveur et la gestion des erreurs

1.4.3 Objectifs organisationnels

- Améliorer la coordination des déplacements domicile-campus au sein de la communauté ENS
- Optimiser l'utilisation des véhicules privés des membres du campus
- Fournir à l'administration un outil de supervision simple et efficace

1.5 Acteurs du système

TABLE 1.1 : Les acteurs du système

Acteur	Rôle	Description
Conducteur	Publication	S'authentifie, publie des trajets, gère ses réservations et reçoit des avis
Passager	Consultation / Réservation	S'authentifie, recherche des trajets, réserve une place et note le trajet
Administrateur	Supervision	Gère les utilisateurs, trajets, réservations et avis via le dashboard web
Système (API)	Traitement	Assure la logique métier, la validation et la communication entre les clients

1.6 Méthodologie adoptée

Pour la réalisation de ce projet, une approche **incrémentale et itérative** a été adoptée, organisée en trois sprints principaux :

- **Sprint 1** — Analyse, conception et API REST : modélisation de la base de données, endpoints CRUD, authentification JWT, tests Postman
- **Sprint 2** — Interface Web React : login, dashboard administrateur, gestion des trajets, réservations et avis
- **Sprint 3** — Application Android : authentification, publication de trajet, recherche, réservation et évaluation

1.6.1 Planification du projet

TABLE 1.2 : Planification des sprints

Sprint	Tâches	Durée
1	Modélisation BDD, API REST, Auth JWT, Tests Postman	1 semaine
2	Interface React : Login, Dashboard, CRUD, Statistiques	1 semaine
3	Android : Authentification, Trajets, Réservations, Avis	2 semaines

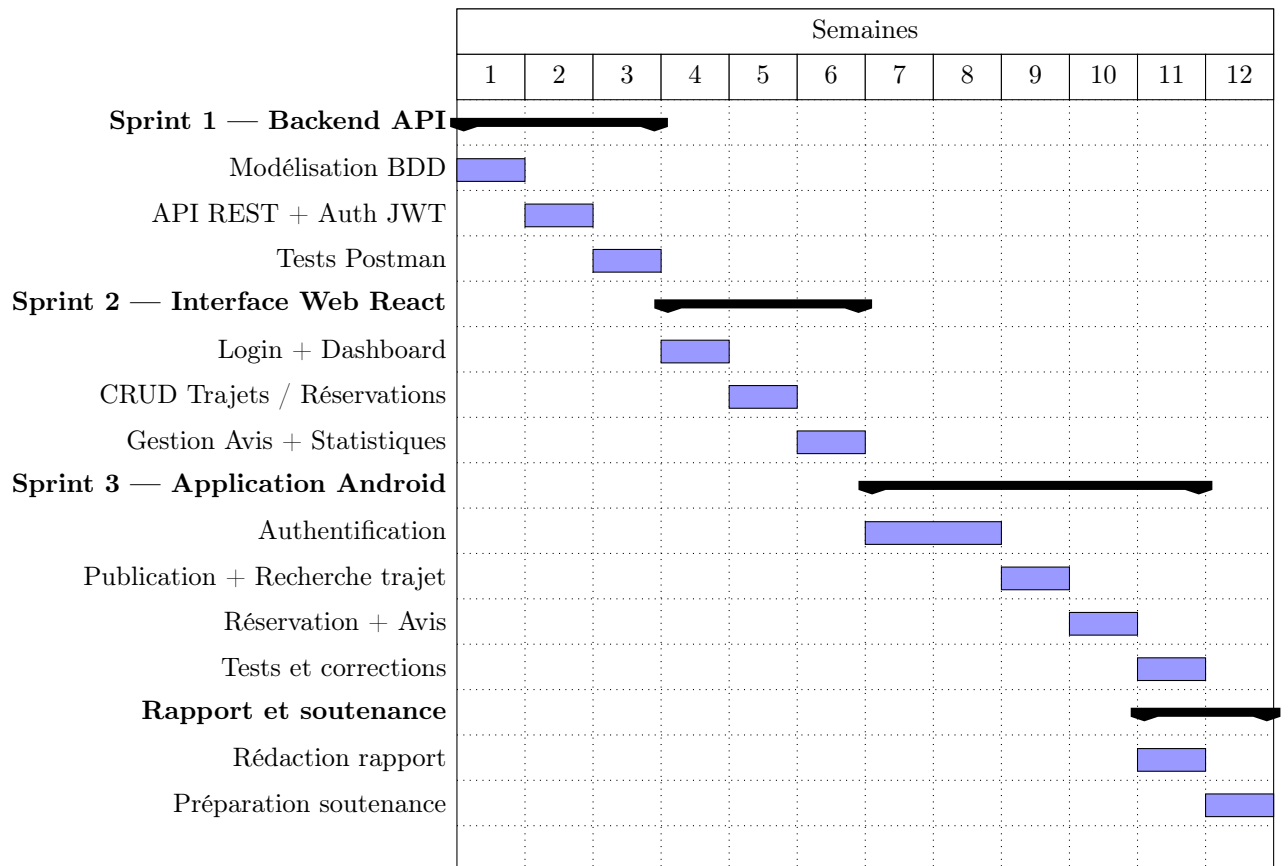


FIGURE 1.1 : Diagramme de Gantt du projet

1.7 Conclusion

Ce chapitre a permis de présenter le contexte du projet de covoiturage urbain ENS Marrakech, la problématique, les objectifs fonctionnels et techniques ainsi que la méthodologie adoptée. Ces éléments constituent une base essentielle pour la suite du travail.

Chapitre 2

État de l’art

2.1 Introduction

Ce chapitre présente une étude des solutions existantes dans le domaine du covoiturage et de la mobilité urbaine. Cette analyse permet d’identifier les forces et limites des systèmes actuels afin de justifier les choix effectués dans le cadre du projet.

2.2 Le covoiturage urbain : définition et enjeux

Le covoiturage désigne le partage d’un véhicule privé entre un conducteur et un ou plusieurs passagers effectuant un trajet commun, généralement pour en réduire le coût et l’impact environnemental [?]. Dans le contexte universitaire, il répond à un besoin particulier : relier régulièrement un domicile à un campus distant, à des horaires récurrents et prévisibles.

Selon une étude de l’ADEME [?], le covoiturage courte distance (moins de 80 km) reste sous-exploité malgré une demande croissante, principalement faute d’outils adaptés au contexte local.

2.3 Solutions existantes

2.3.1 Applications grand public de covoiturage

- **BlaBlaCar** [?] — Leader mondial du covoiturage longue distance. Non adapté aux trajets quotidiens domicile-campus : absence de filtrage par communauté fermée, frais de plateforme, interface non adaptée à un usage récurrent.
- **Karos** — Application de covoiturage domicile-travail. Présente l’approche la plus proche du projet, mais ne permet pas la personnalisation pour une institution et n’est pas disponible au Maroc.
- **Klaxit** — Solution de covoiturage domicile-travail avec gestion employeur. Coûteuse et conçue pour les entreprises, inadaptée à un établissement universitaire public.

2.3.2 Applications de transport urbain

- **Google Maps / Waze** — Solutions de navigation généralistes non conçues pour la mise en relation conducteur-passager.
- **Heetch / InDrive** — Applications de VTC qui impliquent une tarification variable et une intermédiation commerciale absente du modèle covoiturage.

2.4 Comparaison des solutions

TABLE 2.1 : Positionnement du projet par rapport aux solutions existantes

Critère	BlaBlaCar	Karos	Klaxit	Notre projet	
Trajets quotidiens domicile-campus	Partiel				
Communauté fermée (campus)	×	×			
Interface d'administration	×	×			
Gestion des réservations					
Système d'avis			×		
Disponible au Maroc		×	×		
Gratuit / Open source	×	×	×		

2.5 Limites des solutions existantes

L'analyse des solutions disponibles révèle plusieurs lacunes communes :

- Absence de système de gestion pour une communauté universitaire fermée
- Coût élevé des solutions professionnelles
- Impossibilité de personnaliser pour le contexte ENS Marrakech
- Pas d'interface d'administration dédiée à un responsable institutionnel
- Non disponibilité de certaines solutions au Maroc

2.6 Positionnement du projet

Le projet proposé se distingue par une approche sur mesure, adaptée aux besoins spécifiques de la communauté ENS Marrakech :

- Application mobile Android pour conducteurs et passagers du campus
- Interface web d'administration pour superviser les trajets, réservations et avis
- Base de données structurée autour de 4 tables : *users*, *rides*, *ride_bookings*, *ride_reviews*
- API REST sécurisée avec JWT, sans coût de licence
- Gestion complète des rôles (administrateur, conducteur, passager)

2.7 Conclusion

L'état de l'art montre que si des solutions de covoiturage existent, aucune ne répond pleinement aux besoins d'un campus universitaire marocain disposant de ressources limitées. Ce constat justifie le développement d'une solution open source, personnalisée et adaptée au contexte local.

Chapitre 3

Analyse fonctionnelle

3.1 Introduction

Ce chapitre présente l'analyse fonctionnelle du système de covoiturage. Il permet d'identifier les besoins des utilisateurs, les fonctionnalités attendues, les règles métier à respecter, ainsi que les cas d'utilisation du système.

3.2 Identification des acteurs

TABLE 3.1 : Les acteurs du système et leurs fonctionnalités

Acteur	Rôle	Fonctionnalités principales
Conducteur	Publication	Publier, modifier et supprimer des trajets ; consulter et gérer les réservations associées ; recevoir des avis
Passager	Réservation	Rechercher des trajets par ville ; réserver une place ; consulter l'état de ses réservations ; noter un trajet
Administrateur	Supervision	Gérer les utilisateurs, trajets, réservations et avis via le dashboard web ; consulter les statistiques
Système (API)	Traitement	Authentification JWT ; validation des données ; application des règles métier ; communication mobile/web

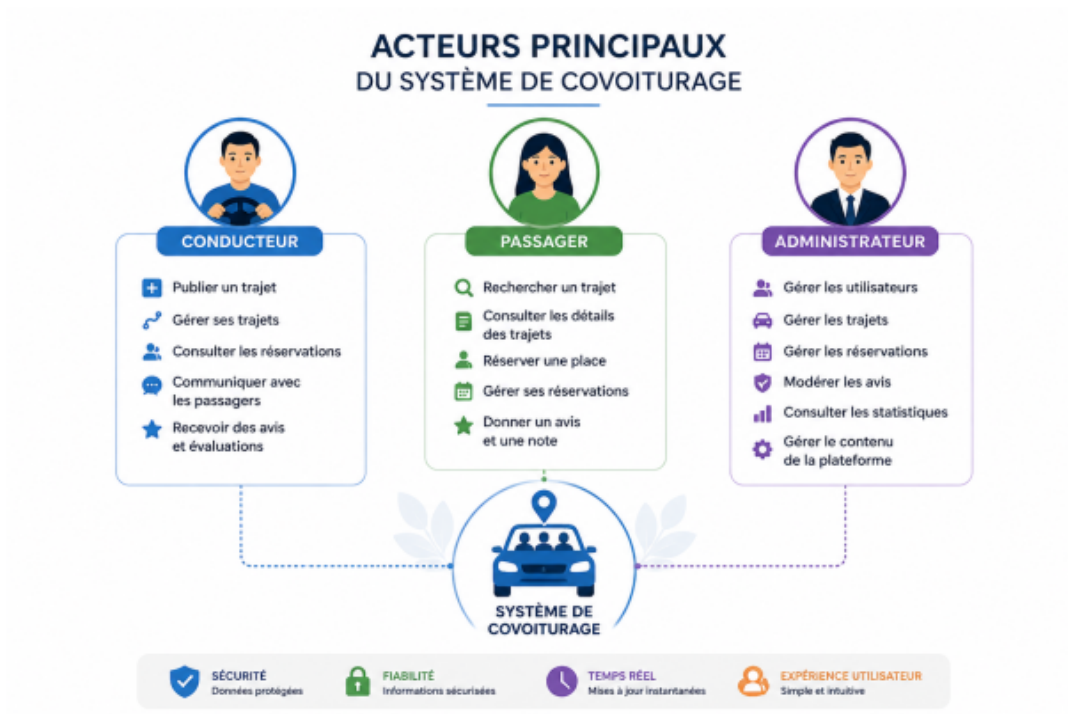


FIGURE 3.1 : Acteurs principaux du système de covoiturage

3.3 Besoins fonctionnels



FIGURE 3.2 : Fonctionnalités principales du système de covoiturage

3.3.1 Besoins du conducteur

Le conducteur est l'acteur qui propose des trajets. Après authentification, il peut :

- Publier un trajet (ville de départ, destination, date, heure, prix, nombre de places)
- Modifier ou supprimer un trajet existant
- Consulter les réservations liées à ses trajets
- Accepter ou refuser des demandes de réservation
- Consulter les avis et notes reçus
- Se déconnecter

3.3.2 Besoins du passager

Le passager recherche et réserve des trajets. Après authentification, il peut :

- Rechercher un trajet par ville de départ
- Consulter les détails d'un trajet disponible
- Réserver une place sur un trajet

- Consulter l'état de ses réservations (en attente, acceptée, refusée)
- Noter un trajet et laisser un commentaire après participation
- Consulter son historique de réservations
- Se déconnecter

3.3.3 Besoins de l'administrateur

L'administrateur supervise le système via l'interface web. Il peut :

- Gérer les utilisateurs (consultation, désactivation de comptes)
- Gérer et modérer les trajets publiés
- Gérer les réservations et leurs statuts
- Modérer les avis et commentaires
- Consulter les statistiques globales via le tableau de bord
- S'authentifier via identifiant et mot de passe

3.3.4 Besoins du système (Backend API)

Le backend assure le fonctionnement global du système :

- Authentification sécurisée via JWT
- Gestion des trajets, réservations et avis
- Contrôle des règles métier (places disponibles, statuts, unicité des réservations)
- Communication entre l'application Android et le dashboard React via API REST
- Validation des données entrantes et gestion des erreurs

3.4 Besoins non fonctionnels

- **Performance** : le système doit répondre aux requêtes en moins de 500 ms
- **Sécurité** : authentification JWT avec expiration 24h, mots de passe hashés avec bcrypt, requêtes SQL préparées contre les injections
- **Disponibilité** : l'API doit rester accessible pendant les heures de cours (7h00–20h00)
- **Ergonomie** : interfaces mobile et web claires, intuitives, adaptées aux étudiants
- **Maintenabilité** : code organisé en modules (routes, controllers, models) pour faciliter les évolutions
- **Scalabilité** : architecture permettant l'ajout futur de fonctionnalités (géolocalisation, notifications push)

3.5 Règles métier

- **R1** : Le nombre de réservations acceptées ne doit pas dépasser le nombre de places disponibles sur un trajet
- **R2** : Un passager ne peut réserver qu'une seule fois le même trajet
- **R3** : Une réservation possède un seul statut actif à la fois : **pending**, **accepted** ou **rejected**
- **R4** : Un utilisateur ne peut noter un trajet que s'il y a effectivement participé (réservation acceptée)
- **R5** : Un trajet complet (plus aucune place disponible) n'accepte plus de nouvelles réservations

— **R6** : Un conducteur ne peut pas réserver sa propre course

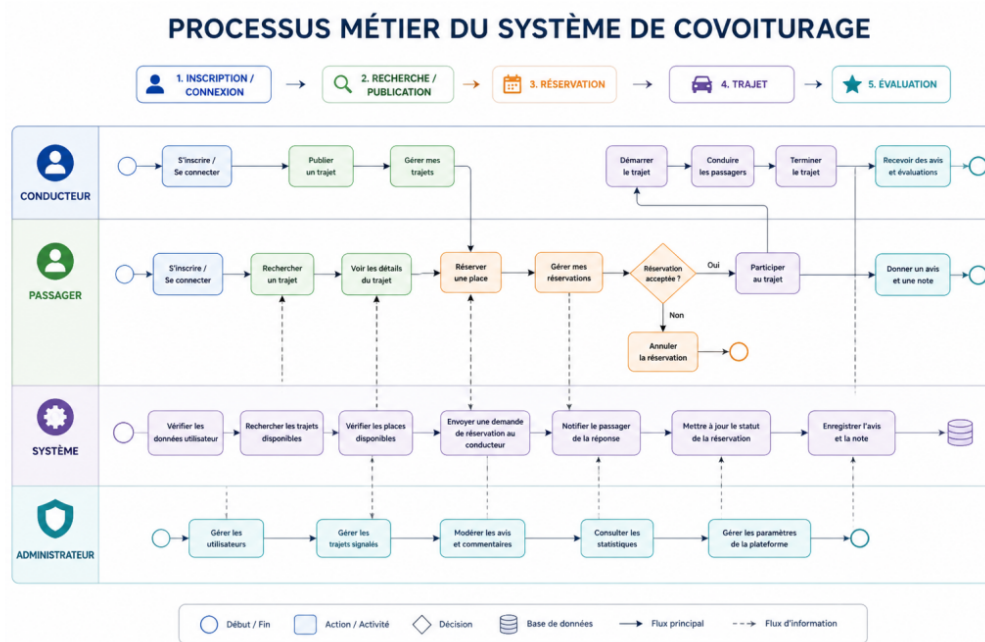


FIGURE 3.3 : Processus métier du système de covoiturage

3.6 Scénarios d'utilisation

3.6.1 Scénario 1 : Publication d'un trajet par un conducteur

Le conducteur s'authentifie sur l'application Android. Il renseigne les informations du trajet : ville de départ (ex. Guéliz), destination (ENS Marrakech), date, heure, prix et nombre de places. L'API valide les données, vérifie que le nombre de places est supérieur à zéro, et crée le trajet dans la table `rides`. Le conducteur reçoit une confirmation.

3.6.2 Scénario 2 : Réservation d'une place par un passager

Le passager recherche un trajet depuis sa ville. La liste des trajets disponibles s'affiche. Il consulte les détails (prix, places restantes, avis du conducteur) et envoie une demande de réservation. L'API vérifie les règles R1, R2 et R5, puis insère la réservation avec le statut `pending`. Le conducteur reçoit la demande et peut l'accepter ou la refuser.

3.6.3 Scénario 3 : Supervision par l'administrateur

L'administrateur se connecte au dashboard web. Il consulte le nombre de trajets actifs, les réservations en attente et les avis signalés. Il peut désactiver un compte, supprimer un trajet problématique ou modérer un avis inapproprié.

3.7 Diagramme de cas d'utilisation

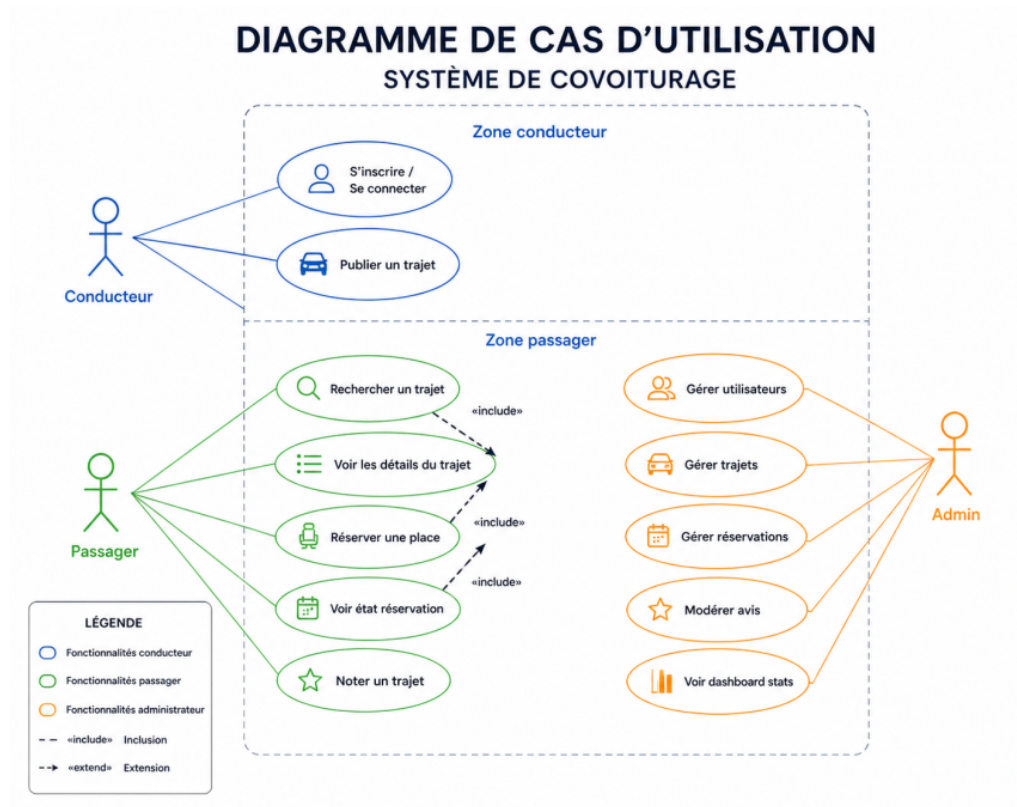


FIGURE 3.4 : Diagramme de cas d'utilisation du système de covoiturage

3.8 Conclusion

Ce chapitre a permis de définir les besoins fonctionnels et non fonctionnels du système de covoiturage, d'identifier les acteurs et leurs rôles, et de formuler les règles métier essentielles. Ces spécifications constituent la base de la phase de conception présentée au chapitre suivant.

Chapitre 4

Conception du système

4.1 Introduction

Ce chapitre présente la conception technique du système de covoiturage. Il définit l'architecture globale, le modèle de données, les principaux diagrammes UML et la conception de l'API REST.

4.2 Architecture générale

Le système repose sur une architecture **client-serveur en trois tiers** :

1. **Tier présentation** — Application Android (Java + Volley) et Interface Web (React)
2. **Tier logique métier** — API REST Node.js/Express avec pattern Routes → Controllers → Database
3. **Tier données** — Base de données MySQL avec 4 tables

La communication entre les clients et le serveur s'effectue via des requêtes HTTP au format JSON. L'authentification est assurée par JWT, transmis dans les en-têtes **Authorization**.

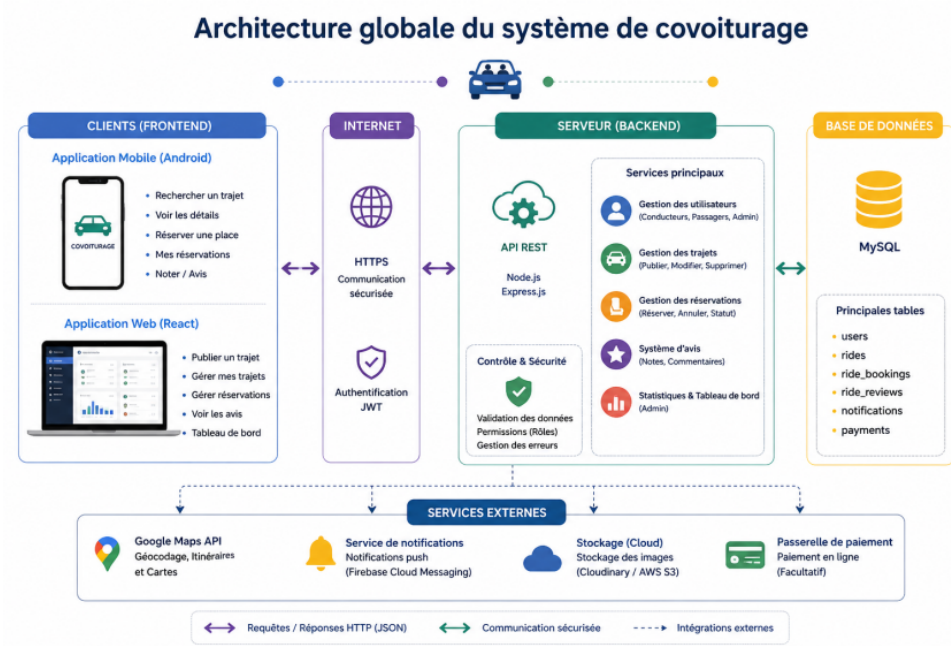


FIGURE 4.1 : Architecture globale du système de covoiturage

4.3 Modèle de données

Le système repose sur quatre tables MySQL, respectant la contrainte du cahier des charges :

TABLE 4.1 : Structure des tables de la base de données

Table	Champ	Description
users	id (PK) name email password role created_at	Identifiant unique Nom complet Adresse email (unique) Mot de passe hashé (bcrypt) driver , passenger ou admin Date d'inscription
rides	id (PK) driver_id (FK) departure_city arrival_city ride_date / time price available_seats status	Identifiant unique Référence vers users Ville de départ Ville d'arrivée Date et heure du trajet Prix par place (DH) Nombre de places disponibles open , full ou cancelled
ride_bookings	id (PK) ride_id (FK) passenger_id (FK) booking_status	Identifiant unique Référence vers rides Référence vers users pending , accepted ou rejected
ride_reviews	id (PK) ride_id (FK) passenger_id (FK) rating comment	Identifiant unique Référence vers rides Référence vers users Note de 1 à 5 Commentaire textuel

Les règles métier R1, R2 et R4 sont appliquées au niveau de l'API : vérification des places disponibles avant insertion dans **ride_bookings**, unicité de la réservation par passager et trajet, et vérification du statut de la réservation avant d'autoriser la création d'un avis dans **ride_reviews**.

4.4 Diagramme de classes

Le diagramme de classes représente la structure statique du système et met en évidence les quatre entités principales ainsi que leurs relations :

- Un **User** (conducteur ou passager) peut publier plusieurs **Rides**
- Un **Ride** peut contenir plusieurs **RideBookings** (une par passager)
- Un **RideBooking** avec statut **accepted** peut générer un **RideReview**

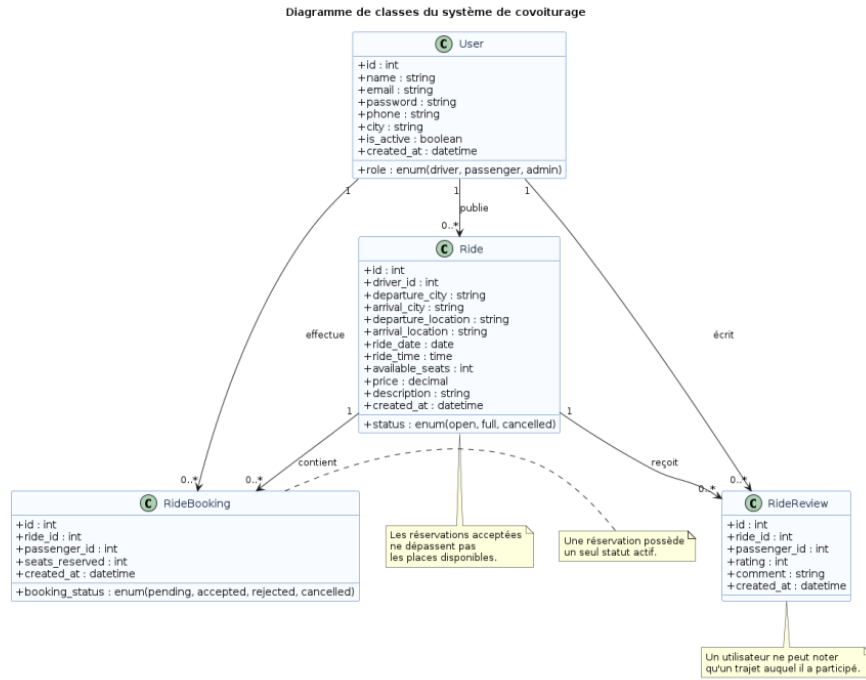


FIGURE 4.2 : Diagramme de classes du système de covoiturage

4.5 Diagrammes de séquence

4.5.1 Réservation d'un trajet

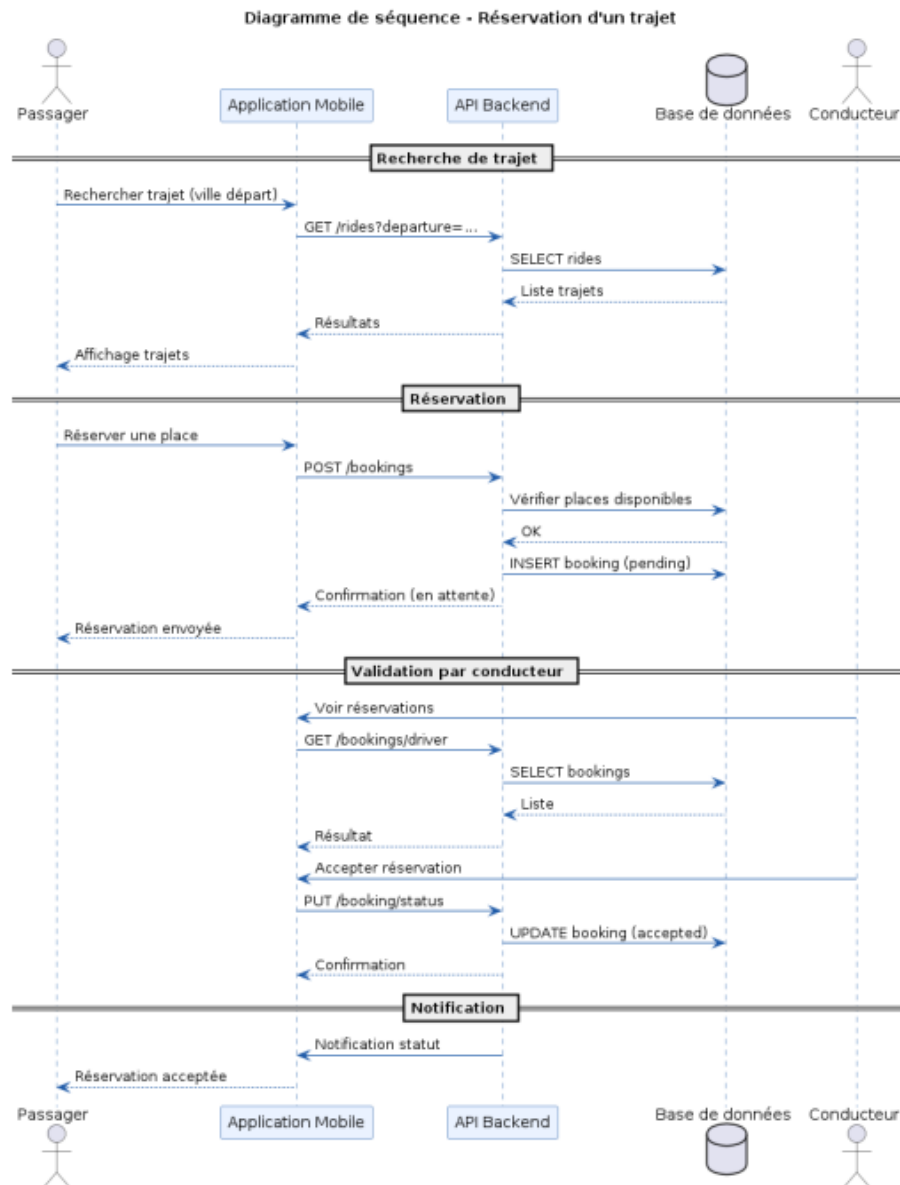


FIGURE 4.3 : Diagramme de séquence de réservation d'un trajet

4.5.2 Réservation d'une place

Ce scénario illustre les interactions lors de la réservation. Le passager envoie une requête `POST /bookings` avec le `ride_id`. L'API vérifie les règles R1, R2 et R5 (places disponibles, unicité, trajet non complet). En cas de succès, la réservation est insérée avec le statut `pending`. Le conducteur consulte les demandes via `GET /bookings/driver` et met à jour le statut via `PUT /booking/status`.

4.6 Conception de l'API REST

TABLE 4.2 : Endpoints API principaux

Endpoint	Méthode	Description
/api/auth/register	POST	Inscription d'un utilisateur
/api/auth/login	POST	Authentification (retourne JWT)
/api/rides	GET	Liste des trajets disponibles
/api/rides	POST	Publier un trajet (conducteur)
/api/rides/:id	PUT/DELETE	Modifier / supprimer (conducteur)
/api/bookings	POST	Réserver une place (passager)
/api/bookings/driver	GET	Réservations de mes trajets (conducteur)
/api/bookings/passenger	GET	Mes réservations (passager)
/api/bookings/:id/status	PUT	Accepter / refuser (conducteur)
/api/reviews	POST	Ajouter un avis (passager)
/api/reviews/:rideId	GET	Avis d'un trajet
/api/admin/users	GET	Liste des utilisateurs (admin)
/api/admin/stats	GET	Statistiques globales (admin)

4.7 Sécurité du système

- Authentification via **JWT** avec expiration de 24 heures
- Hachage des mots de passe avec **bcrypt** (cost factor = 10)
- Requêtes SQL préparées (parameterized queries) contre les injections SQL
- Validation des données entrantes côté serveur avant toute opération
- Middleware de vérification du rôle pour les routes protégées (admin, driver)

4.8 Conclusion

Ce chapitre a présenté la conception du système de covoiturage, incluant l'architecture client-serveur, le modèle de données à 4 tables, les diagrammes UML et la conception de l'API REST. Ces éléments constituent une base solide pour la phase de réalisation décrite au chapitre suivant.

Chapitre 5

Réalisation du projet

5.1 Introduction

Ce chapitre présente la phase de réalisation du projet de covoiturage urbain. Il décrit les technologies utilisées, la structure du code, les interfaces développées et les tests effectués.

5.2 Environnement de développement

- **Android Studio** — IDE principal pour le développement Android (Java)
- **Visual Studio Code** — Développement Node.js et React
- **Git + GitHub** — Gestion de versions et sauvegarde du code source
- **Postman** — Test et validation des endpoints API
- **phpMyAdmin (XAMPP)** — Administration de la base de données MySQL locale

5.3 Choix technologiques

TABLE 5.1 : Technologies utilisées et justification

Composant	Technologie	Justification
Backend API	Node.js + Express	Léger, performant, adapté aux API REST, grande communauté
Base de données	MySQL	SGBD relationnel fiable, adapté aux 4 tables structurées
Interface web	React	Composants réutilisables, rafraîchissement dynamique du dashboard
App mobile	Android Java	Langage natif, accès complet aux fonctionnalités du téléphone
HTTP mobile	Volley	Bibliothèque HTTP légère, intégrée nativement dans l'écosystème Android
Authentification	JWT	Authentification stateless, sécurisée, sans gestion de session serveur

5.4 Réalisation du backend

5.4.1 Structure du projet

```
1 covoiturage-api/  
2     config/db.js          # Connexion MySQL avec pool  
3     controllers/  
4         authController.js  
5         rideController.js  
6         bookingController.js  
7         reviewController.js  
8     middleware/  
9         authMiddleware.js # Verification JWT + role  
10    routes/  
11        authRoutes.js  
12        rideRoutes.js  
13        bookingRoutes.js  
14        reviewRoutes.js  
15    .env  
16    server.js
```

Listing 5.1 – Structure du backend Node.js

5.4.2 Middleware d'authentification JWT

```
1 const verifyToken = (req, res, next) => {  
2     const token = req.headers['authorization']?.split(' ')[1];  
3     if (!token) return res.status(401).json({  
4         error: 'Acces refuse. Token manquant.'  
5     });  
6     try {  
7         req.user = jwt.verify(token, process.env.JWT_SECRET);  
8         next();  
9     } catch {  
10        res.status(403).json({ error: 'Token invalide.' });  
11    }  
12 };
```

Listing 5.2 – Middleware JWT (authMiddleware.js)

5.5 Réalisation de l'interface web (Dashboard)

5.5.1 Page de connexion

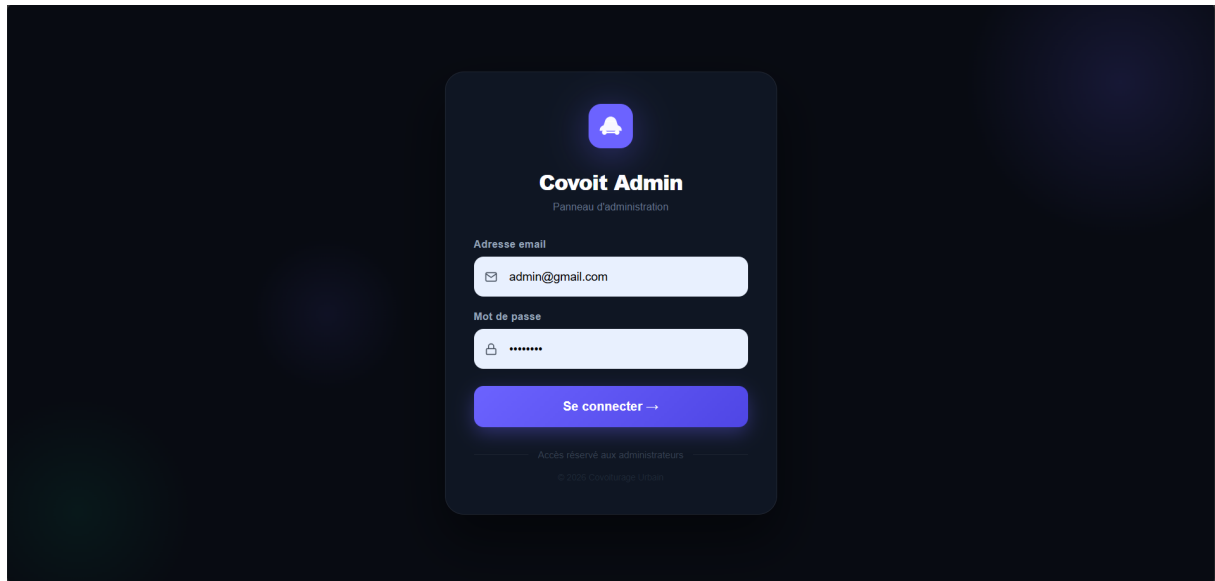


FIGURE 5.1 : Dashboard administrateur — Page de connexion

5.5.2 Tableau de bord

Le tableau de bord affiche en temps réel les indicateurs principaux du système :

- Nombre total de trajets publiés
- Nombre de réservations (total et par statut)
- Note moyenne des avis
- Répartition des statuts de réservation (graphique)

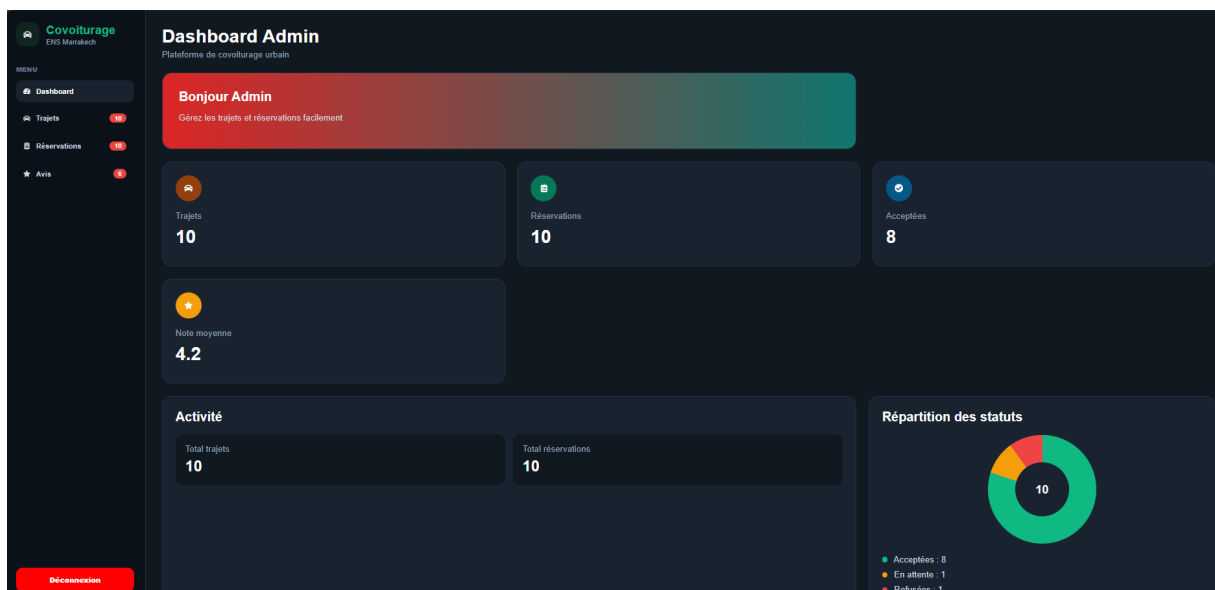


FIGURE 5.2 : Dashboard administrateur — Vue générale des statistiques

5.5.3 Gestion des trajets

L'interface de gestion des trajets permet à l'administrateur de consulter tous les trajets publiés avec leurs informations (ville de départ, ville d'arrivée, prix, places disponibles, statut). Il peut filtrer par ville et supprimer un trajet.

Départ	Arrivée	Prix	Places
Marrakech	Fes	250.00 DH	3
Tiznit	Tanger	130.00 DH	1
Marrakech	Essaouira	180.00 DH	1
Laayoune	Safi	35.00 DH	2
Inzgane	Taroudante	70.00 DH	1
Rabat	Meknes	225.00 DH	1
Rabat	Tanger	100.00 DH	1
Laayoune	Rabat	10.00 DH	2
Safi	Agadir	120.00 DH	1
Rabat	Agadir	100.00 DH	2

FIGURE 5.3 : Dashboard administrateur — Liste des trajets

5.5.4 Gestion des réservations

L'interface de gestion des réservations affiche les demandes avec leur statut (`pending`, `accepted`, `rejected`), le passager et le trajet concernés. Le filtrage par ville de départ est disponible.

Passager	Départ	Arrivée	Statut	Actions
khalid makboul	Rabat	Agadir	accepted	✓ ✕
khalid makboul	Rabat	Tanger	pending	✓ ✕
khalid makboul	Safi	Agadir	accepted	✓ ✕
Sara Passenger	Rabat	Tanger	accepted	✓ ✕
samadi el	Rabat	Meknes	accepted	✓ ✕
Sara Passenger	Marrakech	Fes	accepted	✓ ✕
Majjati moahmed taha	Tiznit	Tanger	accepted	✓ ✕
Omar Passenger	Marrakech	Essaouira	accepted	✓ ✕

FIGURE 5.4 : Dashboard administrateur — Gestion des réservations

5.5.5 Gestion des avis

L'interface des avis permet à l'administrateur de consulter les notes et commentaires laissés par les passagers. Les avis sont affichés avec la note (étoiles), le commentaire, le passager et le

trajet concerné. La modération permet de supprimer un avis inapproprié.

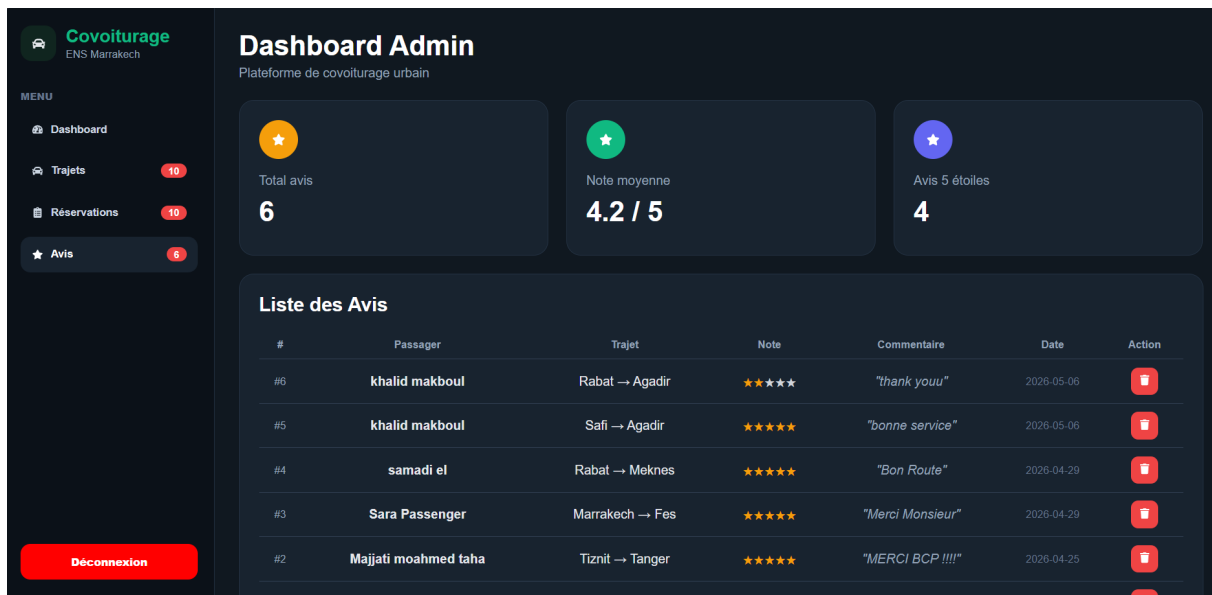


FIGURE 5.5 : Dashboard administrateur — Gestion des avis

5.6 Réalisation de l'application mobile

5.6.1 Structure du projet Android

```

1 app/src/main/java/com/example/covoiturage/
2     api/
3         ApiConfig.java          # URL de base de l'API
4         VolleyHelper.java       # Requetes HTTP avec Volley
5     models/                     # Classes de donnees (POJO)
6         Ride.java
7         Booking.java
8         Review.java
9     utils/
10        SessionManager.java     # Gestion JWT local (SharedPreferences)
11    ui/
12        auth/
13            LoginActivity.java
14            RegisterActivity.java
15        driver/
16            PublishRideActivity.java
17            MyRidesActivity.java
18        passenger/
19            SearchRideActivity.java
20            BookingActivity.java
21            MyBookingsActivity.java

```

Listing 5.3 – Structure de l'application Android

5.6.2 Écran de connexion

L'écran d'authentification permet aux utilisateurs de se connecter avec leur email et mot de passe. Le rôle (conducteur ou passager) est sélectionné lors de l'inscription. L'API retourne un JWT qui est stocké localement via `SharedPreferences`.

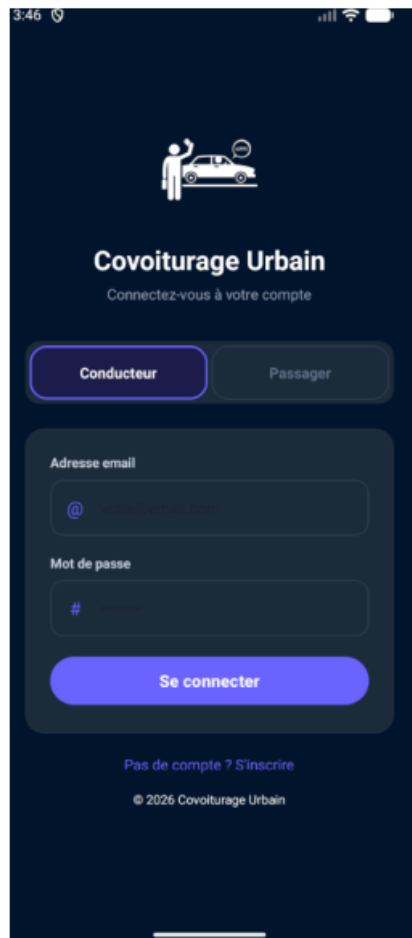


FIGURE 5.6 : Application Android — Écran de connexion

5.6.3 Publication d'un trajet (conducteur)

L'interface de publication permet au conducteur de renseigner les informations du trajet. La validation des champs est effectuée côté client (prix > 0 , places ≥ 1 , date future) avant l'envoi à l'API via `POST /api/rides`.

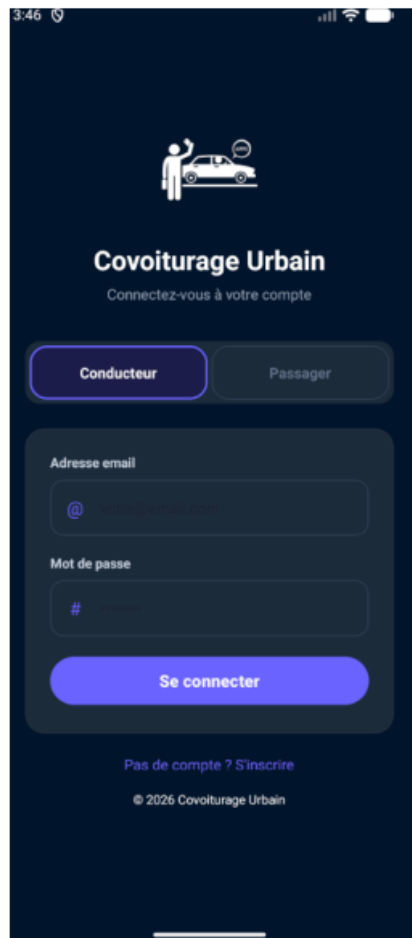


FIGURE 5.7 : Application Android — Publication d'un trajet

5.6.4 Recherche et réservation (passager)

L'interface de recherche affiche la liste des trajets disponibles filtrés par ville de départ. Le passager consulte les détails (conducteur, prix, places restantes, note moyenne) et peut réserver une place. Le statut de sa réservation est consultable depuis l'historique.



FIGURE 5.8 : Application Android — Recherche et réservation d'un trajet

5.7 Tests et validation

TABLE 5.2 : Résultats des tests API (Postman)

Endpoint	Méthode	Résultat	Cas testés
/api/auth/login	POST		Login valide, mot de passe incorrect
/api/rides	POST		Trajet valide, champs manquants
/api/bookings	POST		Réservation valide, trajet complet, doublon
/api/bookings/:id/status	PUT		Accepter, refuser, statut invalide
/api/reviews	POST		Avis valide, sans réservation acceptée
/api/admin/stats	GET		Statistiques globales

5.8 Conclusion

Ce chapitre a présenté la réalisation complète du projet de covoiturage urbain, incluant le backend Node.js, le dashboard React et l'application Android. L'ensemble des fonctionnalités définies dans le cahier des charges ont été implémentées et validées via des tests Postman. Le prototype est fonctionnel en environnement local et prêt pour un déploiement sur serveur.

Conclusion générale

Récapitulation des objectifs

Ce projet avait pour objectif de concevoir et développer une plateforme de covoiturage urbain adaptée aux besoins quotidiens de la communauté de l'École Normale Supérieure de Marrakech, en particulier pour les trajets domicile-campus. L'ensemble des objectifs fonctionnels et techniques définis au départ ont été atteints.

Contributions et résultats

Les réalisations concrètes du projet incluent :

- Une **API REST complète** avec authentification JWT, validation des données et gestion des rôles
- Un **dashboard web React** avec supervision des trajets, réservations, avis et statistiques en temps réel
- Une **application Android** permettant la publication de trajets, la recherche, la réservation et l'évaluation
- Une **base de données MySQL** structurée autour de 4 tables respectant les règles métier définies
- Des **tests API Postman** couvrant les scénarios nominaux et les cas d'erreur

Limites du prototype

- Le système fonctionne en réseau local (XAMPP) — un déploiement en production nécessiterait un serveur avec IP publique
- La recherche de trajets est limitée au filtrage par ville de départ, sans géolocalisation
- Le système de notification n'est pas implémenté dans cette version

Perspectives d'amélioration

- Déploiement sur un serveur cloud pour un accès depuis n'importe quel réseau
- Intégration d'un système de géolocalisation en temps réel via OpenStreetMap
- Ajout de notifications push Android lors de l'acceptation ou du refus d'une réservation
- Intégration d'un système de paiement en ligne pour les frais de trajet
- Extension à d'autres établissements universitaires de la région

Ce projet a constitué une expérience complète de développement full-stack, couvrant l'analyse des besoins, la conception UML, le développement Android, l'architecture REST et la supervision web. Il apporte une solution concrète et moderne à un besoin réel de la communauté ENS Marrakech.



Majjati Mohamed Taha

Covoiturage Urbain — ENS Marrakech

Application Mobile Android & Dashboard React

Ce projet porte sur la conception et le développement d'une plateforme de covoiturage urbain destinée aux étudiants, enseignants et personnels de l'École Normale Supérieure de Marrakech. L'objectif est de faciliter l'organisation des trajets domicile-campus en mettant en relation conducteurs et passagers via une solution numérique simple et fiable.

Le système repose sur une architecture client-serveur comprenant une API REST développée avec Node.js et Express, une base de données MySQL structurée autour de quatre tables (*users*, *rides*, *ride_bookings*, *ride_reviews*), une application web React pour l'administration et une application mobile Android pour les utilisateurs. Il intègre une authentification sécurisée par JWT et une gestion complète des rôles (administrateur, conducteur, passager).

Les fonctionnalités principales incluent la publication et la recherche de trajets, la réservation de places avec suivi des statuts, un système d'avis et de notes, ainsi qu'un tableau de bord administrateur avec statistiques.

Mots-clés : Covoiturage, Android Java, React.js, Node.js, MySQL, API REST, JWT, Dashboard, Volley, UML.

Encadré par : Pr. Mohamed LACHGAR

Institution : École Normale Supérieure de Marrakech — Université Cadi Ayyad

Année : 2025–2026